TP 2 — Pentest guidé de l'application corrigée

Module Sécurité des Smartphones · Séance 5 · Chaymae Noughou
Application testée : **MobileVault** (`edu.lab.mobilevault`), corrigée au TP 1 · Émulateur Pixel 8 (API 35), ADB · Données entièrement fictives

Introduction : méthode du pentest

Le TP 2 reprend l'application corrigée au TP 1 et la soumet à **5 investigations** (P1 à P5). Chaque investigation suit le même déroulé :

1. **Reproduire** la vulnérabilité et garder une **preuve** (avant correction) ;
2. **Corriger** ;
3. Refaire **exactement le même test** : c'est le **contre-test obligatoire** (après correction) ;
4. Nommer la **propriété de sécurité** démontrée.

> **Adaptation :** le guide est écrit pour l'application « SecureBank » (`edu.lab.securebank`, tag `SecureBankLab`, fichier `lab_session.xml`). Ces noms ont été remplacés par `edu.lab.mobilevault`, `MobileVaultLab` et `vault_lab.xml`. Sous Windows PowerShell, `grep` est remplacé par `findstr`.

Vérification initiale : `adb shell pm list packages | findstr mobilevault`

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb shell pm list packages | findstr mobi
levault
package:edu.lab.mobilevault
```

L'application de TP est bien installée sur l'émulateur.

Investigation	Question posée	Propriété démontrée
P1 — Jeton en clair	Le jeton est-il lisible dans le stockage ?	Confidentialité au repos
P2 — Logcat	Le jeton fuit-il dans les journaux ?	Pas de fuite à l'exécution
P3 — Intégrité	Un chiffré modifié est-il rejeté ?	Intégrité (tag GCM)
P4 — Perte de clé	Que se passe-t-il sans la clé ?	Échec contrôlé, résilience
P5 — 401 / 403	Qui décide de la validité et des droits ?	Autorisation côté serveur

Vulnérabilités identifiées et corrections

L'audit du code et les tests ont permis d'identifier **7 vulnérabilités** dans l'application d'origine : un secret embarqué, des données sensibles en clair, un booléen de session, Base64 utilisé comme « protection », un log sensible et une sauvegarde permissive. Pour chacune, ce rapport présente le code d'origine, pourquoi c'est dangereux, la correction appliquée et l'endroit où elle est prouvée.

E1 — Secret (mot de passe) écrit en dur dans le code

📍 MainActivity.kt, ligne 13

```
private const val LAB_PASSWORD = "vault-demo-2026"
...
if (password.text.toString() == LAB_PASSWORD) { ... }
```

❌ **Pourquoi c'est dangereux :** Le mot de passe est dans l'APK : n'importe qui peut le lire en décompilant l'application (jadx, apktool). Le contrôle d'identité est fait **sur le téléphone**, donc il peut être contourné.

✅ **Correction :** La version corrigée de `MainActivity.kt` ne contient plus aucun mot de passe : la connexion est fictive. Dans une vraie application, l'identité est vérifiée **par le serveur**, et aucun secret partagé n'est embarqué dans l'APK.

```
addButton("1. Connexion fictive") { store.save("demo_" + UUID.randomUUID().toString()) ... }
```

Preuve dans ce rapport : P5 (authentification et droits vérifiés côté serveur)

E2 — Jeton de session stocké en clair

📍 MainActivity.kt, ligne 38

```
prefs.edit().putBoolean("logged_in", true).putString("session_token", token).apply()
```

❌ **Pourquoi c'est dangereux :** `MODE_PRIVATE` isole le fichier mais ne le chiffre pas : avec un accès debug (`run-as`), root ou une sauvegarde, le jeton se lit directement, et le voler suffit pour usurper la session.

✅ **Correction :** Nouveau composant `SecureTokenStore` : chiffrement **AES-GCM** (clé AES-256 générée dans **Android Keystore**, IV aléatoire de 12 octets, tag de 16 octets). Seul `[version][IV][chiffré+tag]` est stocké.

```
store.save(token)   // → secure_session.xml : encrypted_token = "Af2N1bcf…" (illisible)
```

Preuve dans ce rapport : P1

E3 — Note privée stockée en clair

📍 MainActivity.kt, ligne 49

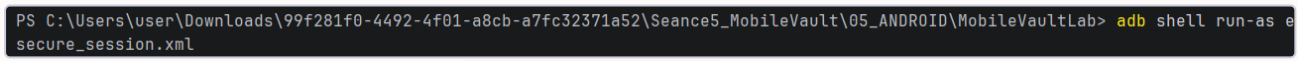
```
prefs.edit().putString("private_note", note.text.toString()).apply()
```

❌ **Pourquoi c'est dangereux :** Même problème que E2 : la donnée « privée » est lisible en clair dans `vault_lab.xml`.

✅ **Correction :** La version corrigée n'enregistre plus de note (si la fonctionnalité était conservée, elle serait chiffrée avec `SecureTokenStore`). **Migration :** au démarrage, l'application supprime l'ancien fichier `vault_lab.xml`, qui contenait encore la note et le jeton en clair. Après correction, `ls shared_prefs` ne montre plus que `secure_session.xml`.

```
store = SecureTokenStore(this)
deleteSharedPreferences("vault_lab")   // migration : supprime l'ancien stockage en clair
```

Preuve dans ce rapport : P1 (avant), capture de migration ci-dessous



Après migration : seul le stockage chiffré `secure_session.xml` reste sur l'appareil ; l'ancien fichier en clair a été supprimé.

E4 — Session représentée par un simple booléen

📍 MainActivity.kt, lignes 34 et 48–57

```
private fun isLoggedIn(): Boolean = prefs.getBoolean("logged_in", false)
...
if (!isLoggedIn()) { show("Accès refusé : session locale absente"); return }
```

❌ **Pourquoi c'est dangereux :** Le contrôle d'accès repose sur une valeur `true/false` modifiable dans le fichier : il suffit de la passer à `true` pour « être connecté ». Rien n'est vérifié.

✅ **Correction :** La session est maintenant dérivée du **déchiffrement réussi** du jeton (`store.load() != null`) : un jeton absent, altéré ou sans clé donne « Aucune session ». La **validité réelle** (expiration, révocation, droits) reste une décision du serveur.

```
val token = store.load()
status.text = if (token == null) "Aucune session (ou donnée invalide)" else "Session locale
restaurée"
```

Preuve dans ce rapport : P5 (401 / 403 décidés par le serveur)

E5 — Base64 présenté comme une protection

📍 VaultCrypto.kt, ligne 7 · bouton « Protéger note (fonction à auditer) »

```
fun protect(plain: String): String =
    Base64.encodeToString(plain.toByteArray(Charsets.UTF_8), Base64.NO_WRAP)
```

❌ **Pourquoi c'est dangereux :** Base64 est un **encodage** : réversible par tout le monde, sans clé (cours 3.1). La note « protégée » se décode en une seconde : c'est une fausse sécurité.

✅ **Correction :** Remplacé par un vrai chiffrement authentifié (AES-GCM + Keystore). Base64 n'est plus utilisé que pour écrire les octets chiffrés sous forme de texte. Le fichier `VaultCrypto.kt` (fausse protection) a été **supprimé du projet**.

```
val ciphertextAndTag = cipher.doFinal(token.toByteArray(Charsets.UTF_8))  // AES-GCM
val encoded = Base64.encodeToString(record, Base64.NO_WRAP)               // simple mise en
texte
```

Preuve dans ce rapport : P1

E6 — Journalisation du jeton (log sensible)

📍 MainActivity.kt, ligne 39

```
Log.d(LOG_TAG, "Debug token=$token")
```

❌ **Pourquoi c'est dangereux :** Le jeton apparaît en clair dans Logcat, lisible par un PC en USB, des outils de debug ou des rapports de plantage, même si le stockage est chiffré.

✅ **Correction :** Log supprimé (TP 1), puis remplacé par un **événement neutre** sans identifiant de session (TP 2).

```
Log.d("MobileVaultLab", "Connexion fictive réussie (session enregistrée)")
```

Preuve dans ce rapport : P2

E7 — Sauvegarde permissive

📍 AndroidManifest.xml, ligne 3 · data_extraction_rules.xml, lignes 3–4

```
android:allowBackup="true"
<cloud-backup><include domain="sharedpref" path="."/></cloud-backup>
<device-transfer><include domain="sharedpref" path="."/></device-transfer>
```

❌ **Pourquoi c'est dangereux :** Toutes les préférences (jeton, note) partent dans les sauvegardes cloud et les transferts vers un autre appareil. Sans sa clé Keystore (qui ne quitte jamais l'appareil), un jeton chiffré restauré est inutilisable ; un jeton en clair, lui, est exposé.

✅ **Correction :** `allowBackup="false"` (versions anciennes) et exclusion des fichiers sensibles dans `data_extraction_rules.xml` (Android 12+).

```
android:allowBackup="false"
<cloud-backup><exclude domain="sharedpref" path="secure_session.xml"/> …</cloud-backup>
<device-transfer><exclude domain="sharedpref" path="secure_session.xml"/> …</device-transfer>
```

Preuve dans ce rapport : P4 (preuve dumpsys : ALLOW_BACKUP disparu)

P1 — Extraction contrôlée du jeton en clair

Pourquoi ce test ? Le stockage privé (`MODE_PRIVATE`) **isole** les applications entre elles, mais il **ne chiffre pas**. Toute personne qui accède au dossier de l'application (mode debug, root, sauvegarde) lit le contenu tel quel.

Procédure : lister puis lire les fichiers de préférences.

```
adb shell run-as edu.lab.mobilevault ls shared_prefs
adb shell run-as edu.lab.mobilevault cat shared_prefs/vault_lab.xml
adb shell run-as edu.lab.mobilevault cat shared_prefs/secure_session.xml
```

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb shell run-as edu.lab.mobilevault ls s
hared_prefs
secure_session.xml
vault_lab.xml
```

Deux fichiers : l'ancien stockage en clair (`vault_lab.xml`) et le nouveau stockage chiffré (`secure_session.xml`).

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb shell run-as edu.lab.mobilevault cat
shared_prefs/vault_lab.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="session_token">DEMO_SESSION_TOKEN_2026</string>
    <boolean name="logged_in" value="true" />
    <string name="private_note">Ma note secrète de test</string>
</map>

PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb shell run-as edu.lab.mobilevault cat
shared_prefs/secure_session.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="encrypted_token">Af2N1bcfgRIsED3rQm+xXoeWRC8LABT5KoQkViExwwos89jMwN8omRW9f8x7kXODZOpWUUofJcBhWFbhqK/kIqtfSdpELw==</string>
</map>
```

Avant (haut) : jeton et note lisibles. Après (bas) : seul `encrypted_token` illisible est stocké. (Dans un rapport officiel, la valeur du jeton doit être masquée : `DEMO_****`.)

- **Chemin exact :** `/data/data/edu.lab.mobilevault/shared_prefs/`
- **Correction :** `SecureTokenStore` du TP 1 (AES-GCM, clé dans Android Keystore).
- **Contre-test :** après une nouvelle connexion, le fichier ne contient qu'une représentation chiffrée.
- **Propriété démontrée : confidentialité au repos**.

> **Observation :** au TP 1, le chiffré commençait par `AZAR8WSN…` ; ici par `Af2N1bcf…`, pour le même type de jeton et la même clé. La différence vient du **nonce (IV) aléatoire**, tiré à chaque chiffrement.

> **Correction complémentaire (migration) :** l'ancien fichier `vault_lab.xml` restait sur l'appareil avec le jeton en clair. L'application le supprime désormais au démarrage (`deleteSharedPreferences("vault_lab")`) : après correction, `ls shared_prefs` ne montre plus que `secure_session.xml` (voir E3).

Note : `run-as` ne fonctionne que parce que l'application est **débogable**. Le test montre la lisibilité du jeton dans le contexte de debug, pas un accès possible par n'importe quelle application.

P2 — Fuite par Logcat

Pourquoi ce test ? Un jeton correctement chiffré au repos peut quand même **fuir pendant la connexion**, par exemple dans les journaux système, lisibles par un PC en USB, des outils de debug ou des rapports de plantage.

```
adb logcat -c                       (vider les journaux)
→ déclencher une connexion dans l'application
adb logcat -d -s MobileVaultLab:D   (lire les journaux de l'application)
```

Avant correction (ancien code : `Log.d(LOG_TAG, "Debug token=$token")`) :

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -c

PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -d -s MobileVaultLab:D
--------- beginning of main
11-02 16:21:26.315 11838 11838 D MobileVaultLab: Debug token=DEMO_SESSION_TOKEN_2026
```

Le jeton apparaît en clair dans Logcat.

Correction : le log sensible est supprimé et remplacé par un **événement neutre**, sans identifiant de session :

```
Log.d("MobileVaultLab", "Connexion fictive réussie (session enregistrée)")
```

Après correction :

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -c

PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -d -s MobileVaultLab:D
--------- beginning of main
11-02 17:10:58.999 13455 13455 D MobileVaultLab: Connexion fictive réussie (session enregistrée)
```

Les journaux fonctionnent toujours, mais aucune valeur de jeton n'apparaît.

Propriété démontrée : pas de fuite de secret à l'exécution. Ce contre-test est plus probant qu'un résultat vide : il prouve que la journalisation marche, mais sans le secret.

P3 — Intégrité AES-GCM

Pourquoi ce test ? AES-GCM est un chiffrement **authentifié** (AEAD) : en plus de chiffrer, il calcule un **tag** qui sert de « sceau ». Un enregistrement modifié doit être **rejeté**, avec l'exception `AEADBadTagException`. Le test utilise le **vrai** AES-GCM du Keystore, et non une simulation.

Implémentation (bouton de debug « 6. Test intégrité », fonction `integrityTestForLab()` dans `SecureTokenStore`) :

```
val record = Base64.decode(prefs.getString(keyName, null), Base64.NO_WRAP)  // vrai enregistrement
relu
fun tryDecrypt(r: ByteArray): String = try {
    cipher.init(Cipher.DECRYPT_MODE, key, GCMParameterSpec(128, r.copyOfRange(1, 13)))
    cipher.doFinal(r.copyOfRange(13, r.size))   // résultat volontairement NON affiché
    "OK (déchiffrement réussi)"
} catch (e: AEADBadTagException) { "REFUS : AEADBadTagException (tag invalide)" }
val intact = tryDecrypt(record)
val modified = record.copyOf(); modified[13] = (modified[13].toInt() xor 1).toByte()  // 1 bit
inversé
val altered = tryDecrypt(modified)
```




```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -d -s MobileVaultLab:D
--------- beginning of main
11-02 17:18:33.418 13723 13723 D MobileVaultLab: Connexion fictive réussie (session enregistrée)
11-02 17:18:35.556 13723 13723 D MobileVaultLab: P3 -> Intact : OK (déchiffrement réussi)
11-02 17:18:35.556 13723 13723 D MobileVaultLab: Modifié : REFUS : AEADBadTagException (tag invalide)
```

Sur le même appareil, avec la même clé : l'enregistrement intact est accepté, l'enregistrement modifié est refusé avec `AEADBadTagException`.

Justification du tag : le tag est calculé avec la clé sur le chiffré. Inverser un seul bit change le tag attendu : au déchiffrement, le tag recalculé ne correspond plus à celui qui est stocké, donc `doFinal()` lève `AEADBadTagException`.

Échec sûr : dans le chemin normal, `load()` attrape l'exception, **efface la session corrompue** et renvoie `null`, ce qui force une reconnexion. Il ne renvoie **jamais une chaîne vide** qui pourrait être prise pour un jeton valide.

Propriété démontrée : intégrité.

P4 — Perte de clé et restauration

Pourquoi ce test ? Les clés du Keystore **ne quittent jamais l'appareil** et ne sont **pas copiées dans les sauvegardes**. Si le fichier chiffré est restauré sur un autre téléphone, ou si la clé est invalidée (changement du verrouillage, de la biométrie), l'ancien chiffré devient **indéchiffrable pour toujours**. L'application ne doit alors ni **planter en boucle**, ni **régénérer une clé en silence**.

Implémentation (bouton « 7. Perte de clé », fonction `keyLossTestForLab()`) : suppression de l'alias seul, puis tentative de lecture sans recréer de clé. Le `catch` de `load()` trace l'échec, sans le secret.

```
keyStore().deleteEntry(alias)        // supprime UNIQUEMENT l'alias de la clé
val result = load()                  // load() ne régénère PAS de clé
// dans load() : catch (ex: Exception) { Log.d(..., "load() refusé : ${ex.javaClass.simpleName}");
clear(); return null }
```




```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -d -s MobileVaultLab:D
--------- beginning of main
11-02 17:23:41.015 13931 13931 D MobileVaultLab: Connexion fictive réussie (session enregistrée)
11-02 17:23:42.481 13931 13931 D MobileVaultLab: load() refusé : IllegalStateException - Clé Keystore absente
11-02 17:23:42.494 13931 13931 D MobileVaultLab: P4 -> Clé supprimée → load() = null (refus contrôlé)
11-02 17:23:42.494 13931 13931 D MobileVaultLab: Enregistrement effacé : true
```

Trace de l'échec contrôlé : clé absente, `load()` renvoie `null`, l'enregistrement irrécupérable est effacé, et l'application ne plante pas.

Contre-test (récupération) : nouvelle connexion, puis restauration de la session.

```
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -c
PS C:\Users\user\Downloads\99f281f0-4492-4f01-a8cb-a7fc32371a52\Seance5_MobileVault\05_ANDROID\MobileVaultLab> adb logcat -d -s MobileVaultLab:D
--------- beginning of main
11-02 17:25:25.487 13931 13931 D MobileVaultLab: Connexion fictive réussie (session enregistrée)
```

Une nouvelle session fonctionne (une nouvelle clé est créée) ; l'ancienne est abandonnée.

Correction : exclure le jeton des sauvegardes

Le jeton chiffré ne sert à rien sans sa clé et ne doit pas sortir du téléphone. Deux modifications :

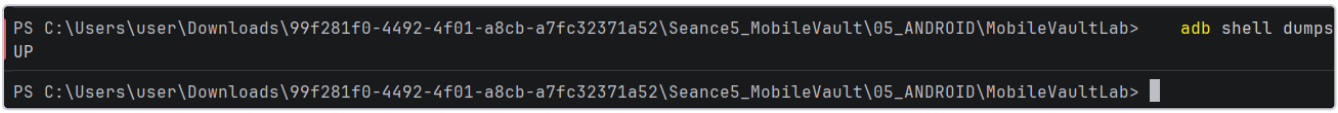
```
AndroidManifest.xml :   android:allowBackup="true"  →  android:allowBackup="false"

data_extraction_rules.xml (Android 12+) :
 <cloud-backup>
  <exclude domain="sharedpref" path="secure_session.xml"/>
  <exclude domain="sharedpref" path="vault_lab.xml"/>
 </cloud-backup>
 <device-transfer> … mêmes exclusions … </device-transfer>
```

Vérification : `adb shell dumpsys package edu.lab.mobilevault | findstr ALLOW_BACKUP`



Avant : le drapeau `ALLOW_BACKUP` est présent, la sauvegarde est autorisée.



Après réinstallation : la commande ne renvoie plus rien, le drapeau a disparu.

> **Erreur rencontrée :** l'attribut `android:exported` de l'activité principale avait été mis à `false` par erreur. L'activité de lancement doit rester `exported="true"`, sinon le build échoue. Seul `allowBackup` devait changer.

Propriété démontrée : échec contrôlé et résilience ; le secret ne quitte plus l'appareil.

P5 — Session expirée et autorisation

Pourquoi ce test ? Un jeton bien chiffré sur le téléphone ne garantit **ni la validité de la session ni les droits d'accès**. C'est le **serveur** qui décide. Deux contrôles distincts :

Code	Contrôle	Question	Réaction attendue du client
401 Unauthorized	Authentification (session)	« Ta session est-elle valide ? »	Effacer ou renouveler la session, demander une reconnexion
403 Forbidden	Autorisation (propriété)	« As-tu le droit sur cette ressource ? »	Afficher un refus, garder la session, ne pas changer les droits

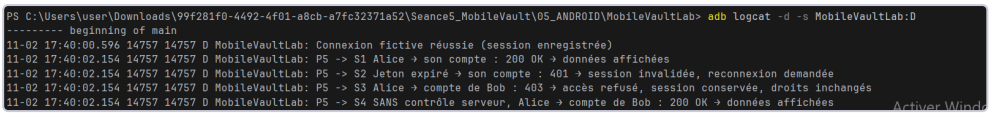
Diagramme du flux

```
Android ──(requête + jeton)──▶ API
                                 │
                  ① Contrôle de session ── absente / expirée / révoquée ──▶ 401
                                 │ OK
                  ② Contrôle de propriété ── ressource d'autrui ──▶ 403
                                 │ OK
                                 ▼
                          200 + données
```

Implémentation : l'application n'a **pas de vrai serveur**. Un **faux serveur local** (`FakeServer.kt`) est donc utilisé : il gère les sessions et leur durée de validité, et vérifie que le compte demandé appartient à l'utilisateur. La fonction `handleResponse()` du client réagit à chaque code. Le bouton « 8. Scénarios serveur » rejoue 4 scénarios.

```
fun getAccount(token: String, accountId: String): Int {
    val s = sessions[token] ?: return 401                                  // ① session inconnue
    if (System.currentTimeMillis() >= s.expiresAt) return 401              // ① session expirée
    if (authorizationCheckEnabled && accountOwner[accountId] != s.userId) return 403 // ② pas
propriétaire
    return 200
}
```





Les 4 scénarios rejoués : 200, 401, 403, puis 200 quand le contrôle serveur est retiré.

Tableau requête / réponse

Scénario	Requête	Réponse	Réaction du client
S1	Alice (session valide) → compte d'Alice	200	Données affichées
S2	Jeton expiré → compte d'Alice	401	Session invalidée, reconnexion demandée
S3	Alice (session valide) → compte de Bob	403	Accès refusé, session conservée, droits inchangés
S4	Même requête que S3, **contrôle ② retiré**	**200**	**Données de Bob renvoyées : faille**

> **Leçon du scénario S4 :** sans contrôle d'autorisation côté serveur, une session **valide** suffit pour lire les données d'un autre utilisateur. **Cacher le bouton** dans l'application Android ne protège rien : un attaquant peut envoyer la requête lui-même (avec un outil comme curl ou Burp, ou une application modifiée). De même, un champ `role` stocké côté mobile n'est **jamais une preuve**.

Correction : appliquer l'autorisation (contrôle de propriété) **côté serveur** sur chaque requête, avec une politique d'expiration et de révocation des jetons.

Propriété démontrée : l'autorisation relève du serveur. Cette démonstration utilise un mock explicite : ce n'est pas une preuve d'autorisation réseau en production.

Bilan du pentest

	Avant	Après
P1 Stockage	Jeton et note en clair	Jeton chiffré AES-GCM, clé dans le Keystore
P2 Logcat	`Debug token=…`	Événement neutre, sans secret
P3 Intégrité	—	Intact accepté, modifié refusé (`AEADBadTagException`)
P4 Clé perdue	Sauvegarde autorisée (`ALLOW_BACKUP`)	Échec contrôlé, récupération, sauvegardes exclues
P5 Serveur	Faille si le contrôle ② manque	401 et 403 distincts ; autorisation côté serveur

Récapitulatif : vulnérabilités → corrections

N°	Vulnérabilité	Statut	Où c'est prouvé
E1	Secret (mot de passe) écrit en dur dans le code	**Corrigé**	P5 (authentification et droits vérifiés côté serveur)
E2	Jeton de session stocké en clair	**Corrigé**	P1
E3	Note privée stockée en clair	**Corrigé**	P1 (avant), capture de migration ci-dessous
E4	Session représentée par un simple booléen	**Corrigé**	P5 (401 / 403 décidés par le serveur)
E5	Base64 présenté comme une protection	**Corrigé**	P1
E6	Journalisation du jeton (log sensible)	**Corrigé**	P2
E7	Sauvegarde permissive	**Corrigé**	P4 (preuve dumpsys : ALLOW_BACKUP disparu)

Toutes les vulnérabilités identifiées sont corrigées et contre-testées.

Limites et éléments de laboratoire à retirer en production

- Boutons et fonctions de test : `tamperForLab()`, `integrityTestForLab()`, `keyLossTestForLab()`, `encryptedRecordForLab()`, l'expiration simulée et le `FakeServer`.
- L'application est **débogable** (`DEBUGGABLE`) : c'est ce qui permet `run-as`. Une application publiée ne doit pas l'être.
- Le Keystore protège la **clé** mais pas son **usage** : sur un appareil compromis, un attaquant peut faire déchiffrer des données pendant que l'application tourne.

> **Conclusion :** chaque vulnérabilité a été reproduite, corrigée puis contre-testée avec la même commande. Le stockage, les journaux, l'intégrité et la gestion de la perte de clé sont désormais maîtrisés côté mobile. P5 rappelle que la sécurité finale (validité de la session, droits d'accès) repose toujours sur le serveur.